

ECE 315

Steven Knudsen, PhD, PEng

Lecture 17 :

- SPI and FreeRTOS
- RP2040 example



UNIVERSITY
OF ALBERTA

Last Lecture

- Serial Peripheral Interface – SPI

This Lecture

- SPI and FreeRTOS
- RP2040 example
- Use Wokwi and make a custom chip, and analogue-to-digital converter (ADC)

Learning Objectives

- Explain the purpose and structure of the Serial Peripheral Interface (SPI) protocol.
- Describe how SPI is used for communication between microcontrollers and peripheral devices.
- Interpret SPI signals (SCK, MISO, MOSI, CS) using waveform visualization tools
 - Using Wokwi logic analyzer block and external viewer
- Understand how to use tools like Wokwi to prove a design before committing to hardware, and why this is a good approach
- Appreciate the limits of using simulation/emulation compared to actual hardware
- Appreciate how having requirements aids in the design process

SPI and ECE 315

- In Lab #3, SPI is explored using a Zynq processor running FreeRTOS
- Today we will look again at FreeRTOS on the RP2040 (Pico) and SPI
- Using Wokwi, we will define a custom SPI device that mimics a 14-bit analogue-to-digital converter (ADC)
- Our objective are to
 - Explore the implementation of SPI in a practical scenario
 - Appreciate and understand the specification of an embedded system
 - Model the system before committing to design
 - Use software tools before committing to hardware (why is this important?)



5

This lecture is more pragmatic than most.

That is, we are treating the material here more like a problem you might encounter in industry than an academic learning exercise.

Why this example and why Wokwi?

- SPI, I2C, and UART are the most common serial protocols you will find in an embedded environment
- Sensors, actuators, and simple input/output devices (switches, joysticks, potentiometers, encoders) are all down in the embedded space
- Need to know how to look at the signals to be sure things are working as design
- Plus... we will have assignments that are based on Wokwi
- It's a great prototyping environment(!)

System Requirements

- Used for “slow” sensors, like temperature
- Sample rate up to 10 samples per second
- High temporal latency and high jitter can be tolerated
 - What quantifies both?
- Factory calibration is all that is needed
 - What does that mean?
- SPI is the preferred interface

ADC Requirements

- 16-bit SPI interface (16-bit word data)
- 14-bit resolution
- ADC samples are in the least significant bits of the 16-bit word

Some Assumptions...

- Choose RP2040 as the most practical, cost-effective embedded processor
- Over specify the temperature measurement resolution
- Assume an ideal temperature sensor
 - Doesn't change with age, is linear in temperature, is inexpensive, is analogue, etc.
- For other reasons, an RTOS is needed
 - Could implement the RP2040 code as bare metal, but won't
- Only functional and performance requirements matter at this point

Pico FreeRTOS template

- Start with the “ECE 315 FreeRTOS Template”
 - Link on Canvas, but also
 - <https://wokwi.com/projects/447824416336657409>
- Save a copy of it naming it “ECE 315 Lecture 15” or something else you like
 - Since this is an example, it can be unlisted



A screenshot of the "Save a Copy" dialog box in the Wokwi web interface. The dialog has a dark gray background. At the top, it says "Save a Copy". Below that is a text input field labeled "Project Name" containing the text "ECE 315 Lec 15". Underneath is the "Project privacy:" section with two radio button options: "Public" (which is unselected) and "Unlisted" (which is selected). Below the radio buttons is a small disclaimer: "Don't share sensitive data (such as passwords) in public projects." At the bottom right, there are two buttons: "CANCEL" and "SAVE COPY".

10

Pico FreeRTOS template

- Start with the “ECE 315 FreeRTOS Template”
 - Link on Canvas, but also
 - <https://wokwi.com/projects/447824416336657409>
- Save a copy of it naming it “ECE 315 Lecture 15” or something else you like
 - Since this is an example, it can be unlisted
 - ***For assignments, make it public so that a link to it can be submitted***

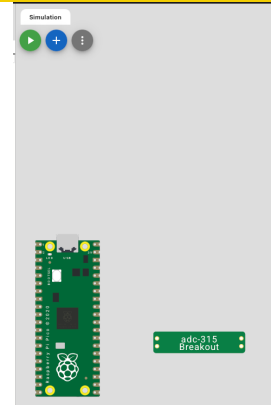


A screenshot of the "Save a Copy" dialog box in the Wokwi web interface. The dialog has a dark grey background. At the top, it says "Save a Copy". Below that is a text input field labeled "Project Name" containing the text "ECE 315 Lec 15". Underneath is a section for "Project privacy:" with two radio button options: "Public" (which is unselected) and "Unlisted" (which is selected). Below the radio buttons is a small note: "Don't share sensitive data (such as passwords) in public projects." At the bottom right of the dialog are two buttons: "CANCEL" and "SAVE COPY".

11

Add Custom Chip – Our SPI ADC

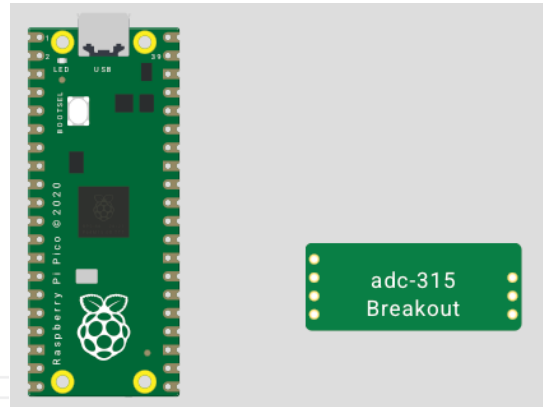
- Click on the Pico and press “r” to rotate it so the USB connector is at the top
- Click the add component button and select “Custom Chip” – name it something like “adc-315”
- Two new files appear at the top left
 - `adc-315.chip.json` and `adc-315.chip.c`
- They are templates we can change to specify the pins and pin functions for our new chip and how it behaves



Define the pins

- Edit the adc-315.chip.json file
- Need power, analogue input, and SPI

```
1 {  
2   "name": "adc-315",  
3   "author": "Steven Knudsen",  
4   "pins": [  
5     "VCC",  
6     "GND",  
7     "IN",  
8     "OUT"  
9   ],  
10  "controls": []  
11 }  
1  
2 {  
3   "name": "adc-315",  
4   "author": "Steven Knudsen",  
5   "pins": [  
6     "SCK",  
7     "MISO",  
8     "MOSI",  
9     "nCS",  
10    "GND",  
11    "A_IN",  
12    "VCC",  
13    ""  
14  ],  
15  "controls": []  
16 }
```



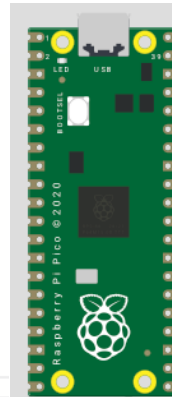
Define the ADC pins

- Edit the adc-315.chip.json file
- Need power, analogue input, and SPI

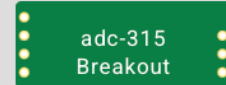
```
1 {  
2   "name": "adc-315",  
3   "author": "Steven Knudsen",  
4   "pins": [  
5     "VCC",  
6     "GND",  
7     "IN",  
8     "OUT"  
9   ],  
10  "controls": []  
11 }
```



```
1 {  
2   "name": "adc-315",  
3   "author": "Steven Knudsen",  
4   "pins": [  
5     "SCK",  
6     "MISO",  
7     "MOSI",  
8     "nCS",  
9     "GND",  
10    "A_IN",  
11    "VCC",  
12    ""  
13  ],  
14  "controls": []  
15 }
```

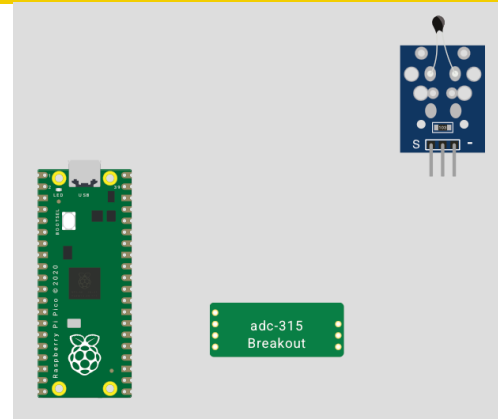


chip1:SCK
chip1:MISO
chip1:MOSI
chip1:nCS
chip1:VCC
chip1:A_IN
chip1:GND



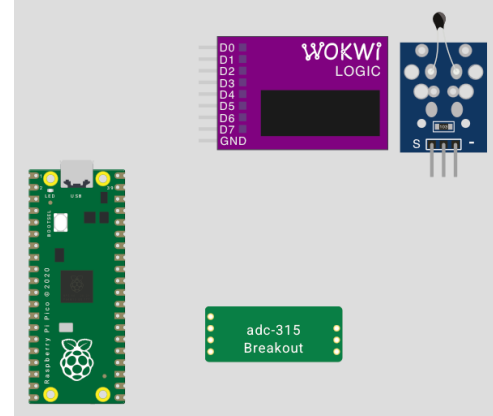
Analogue Signal Source – Thermistor

- Click the add component button and choose the “Analog Temperature Sensor (NTC)”
 - NTC == negative temperature coefficient
 - As temperature goes up, the resistance goes down
- Rotate and place to the right of the other chips



Watching SPI Signals – Logic Analyzer

- Wokwi has a simple logic analyzer that will let us see the SPI signals, so let's add it.
- Now it's time to connect everything...
- Which pins on the Pico to use?

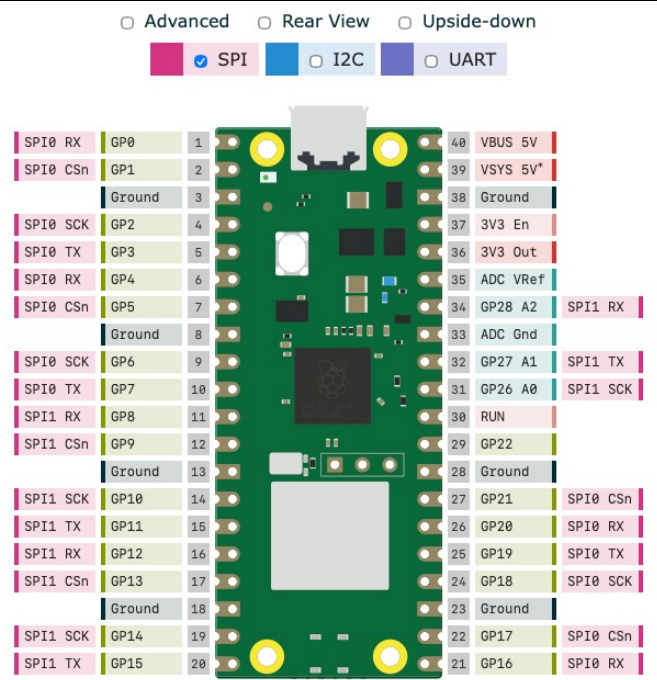


Raspberry Pi Pico W Pinout

- <https://picow.pinout.xyz>
- Use SPI0
 - SPI0 RX (GP16) <-> MISO
 - SPI0 CSn (GP17) <-> nCS
 - SPI0 SCK (GP18) <-> SCK
 - SPI0 TX (GP19) <-> MOSI
- Use VBUS 5V for thermistor and adc-315
- Use GND pins 23 and 38

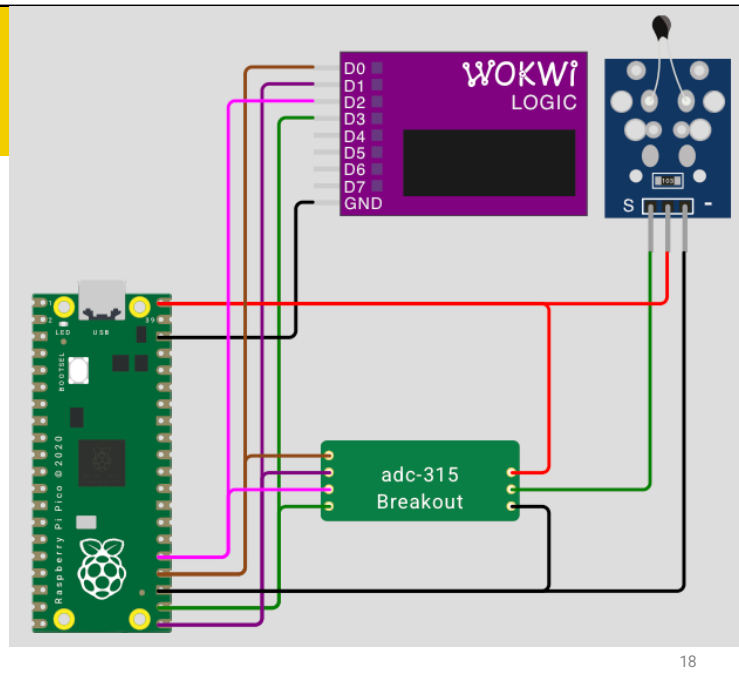


17



Connections

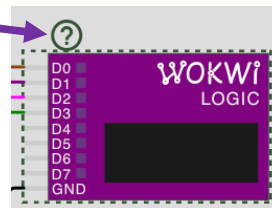
- Last, connect the adc-315 A_IN to the thermistor OUT
- One last thing to do so that the logic analyzer signals match our SPI signals



Logic Analyzer Signal Names

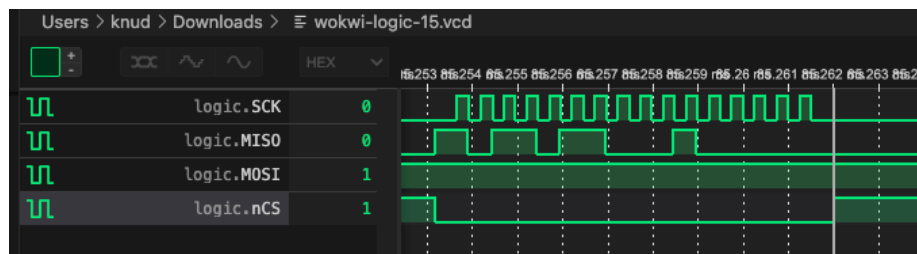
- Change the `diagram.json` file so the `wokwi-logic-analyzer` section has the attributes (`attrs`) shown
- This will embed the right SPI signal names in the file output by the logic analyzer
- To find out more about the logic analyzer, click on it and select the ?
- The file it will output is named `wokwi-logic-*.vcd`, where `*` is a number

```
{  
  "type": "wokwi-logic-analyzer",  
  "id": "logic1",  
  "top": -153.25,  
  "left": 9.6,  
  "attrs": { "channelNames": "SCK,MISO,MOSI,nCS" }  
}
```



Logic Analyzer Waveforms

- When the logic analyzer runs, it outputs a file containing waveform information called a Value Change Dump (VCD)
- This can be viewed using GTKWave, <https://gtkwave.sourceforge.net>
- There is a VSCode plugin, WaveTrace that works pretty well.
- Select a trace and use keyboard arrow keys to find the signal edges



20

Coding the adc-315

- When we created the custom chip in Wokwi, the adc-315, two files were created
 - `adc-315.chip.json` and `adc-315.chip.c`
- We modified the json file to specify the signal and power pins for the custom chip
- Wokwi provides documentation on the creation of custom chips including an SPI guide
 - <https://docs.wokwi.com/chips-api/spi>
- The guides all have examples

adc-315.chip.c

- The template looks like this
- We need to manage the SPI and our analogue signal input
- As a slave device, the chip select signal, nCS, must be monitored to see when it goes low. That indicates the master is talking to us



```
// Wokwi Custom Chip - For docs and examples see:
// https://docs.wokwi.com/chips-api/getting-started
//
// SPDX-License-Identifier: MIT
// Copyright 2025 Steven Knudsen

#include "wokwi-api.h"
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    // TODO: Put your chip variables here
} chip_state_t;

void chip_init() {
    chip_state_t *chip = malloc(sizeof(chip_state_t));

    // TODO: Initialize the chip, set up IO pins, create timers, etc.

    printf("Hello from custom chip!\n");
}
```

22

adc-315.chip.c

- The chip state info the two pins we manage, a reference to an SPI instance (`spi`) and the 16-bit buffer
- Remember, we are making a 14-bit ADC, so the closest buffer size is 2 bytes, 16-bits
- The `chip_pin_change` and `chip_spi_done` declarations are needed to support SPI operation



23

```
// Wokwi Custom Chip - For docs and examples see:
// https://docs.wokwi.com/chips-api/getting-started
//
// SPDX-License-Identifier: MIT
// Copyright 2025 Steven Knudsen

#include "wokwi-api.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

/*
 * ADC scaling
 *
 * Vref is 5 V, which we make the full-scale 14-bit value of 0x3FFF
 */
const float ADC_full_scale = ((float) 0x3FFF) / 5.0;

typedef struct {
    pin_t    a_in_pin;
    pin_t    cs_pin;
    uint32_t spi;
    uint8_t  spi_buffer[2];
} chip_state_t;

static void chip_pin_change(void *user_data, pin_t pin, uint32_t value);
static void chip_spi_done(void *user_data, uint8_t *buffer, uint32_t count);
```

adc-315.chip.c

- The chip initialization
 - Allocates storage for the chip state
 - Initializes the `A_IN` and `nCS` pins
 - Set the `nCS` pin to be watched for both rising and falling edge events
 - Defines the SPI configuration and initializes it
- Note that `chip_pin_change` is a function reference; i.e., a callback function that is called when the `nCS` pin state changes



24

```
void chip_init() {
    chip_state_t *chip = malloc(sizeof(chip_state_t));

    chip->a_in_pin = pin_init("A_IN", ANALOG);

    chip->cs_pin = pin_init("nCS", INPUT_PULLUP);

    const pin_watch_config_t watch_config = {
        .edge = BOTH,
        .pin_change = chip_pin_change,
        .user_data = chip,
    };
    pin_watch(chip->cs_pin, &watch_config);

    const spi_config_t spi_config = {
        .sck = pin_init("SCK", INPUT),
        .miso = pin_init("MISO", INPUT),
        .mosi = pin_init("MOSI", INPUT),
        .done = chip_spi_done,
        .user_data = chip,
    };
    chip->spi = spi_init(&spi_config);

    printf("ADC-315 initialized!\n");
}
```


adc-315.chip.c

- The callback, `chip_pin_change`, starts the SPI read response
 - That is, it outputs on MISO the 16 bits in the `spi_buffer`
 - When complete, it calls `chip_spi_done`, which loads into the buffer the next 14-bit sample



25

```
void chip_pin_change(void *user_data, pin_t pin, uint32_t value) {
    chip_state_t *chip = (chip_state_t*)user_data;
    // Handle nCS pin logic
    if (pin == chip->cs_pin) {
        if (value == LOW) {
            spi_start(chip->spi, chip->spi_buffer, sizeof(chip->spi_buffer));
        } else {
            spi_stop(chip->spi);
        }
    }
}

void chip_spi_done(void *user_data, uint8_t *buffer, uint32_t count) {
    chip_state_t *chip = (chip_state_t*)user_data;
    if (!count) {
        // This means that we got here from spi_stop, and no data was received
        return;
    }

    // Read the analog input value, change to an 14-bit value, save in
    // the buffer.
    // The result will be read back during the next SPI transfer.
    float temp = ADC_full_scale*pin_adc_read(chip->a_in_pin);
    uint16_t itemp = (uint16_t) (temp);
    chip->spi_buffer[0] = itemp & 0x00FF;
    chip->spi_buffer[1] = (itemp >> 8) & 0x00FF;
}
```

adc-315.chip.c

- `pin_adc_read` returns a floating point value between 0 and Vref (5V).
- It is scaled to be from 0 to $2^{14} = 16,383$ and then put into the buffer



26

```
void chip_pin_change(void *user_data, pin_t pin, uint32_t value) {
    chip_state_t *chip = (chip_state_t*)user_data;
    // Handle nCS pin logic
    if (pin == chip->cs_pin) {
        if (value == LOW) {
            spi_start(chip->spi, chip->spi_buffer, sizeof(chip->spi_buffer));
        } else {
            spi_stop(chip->spi);
        }
    }
}

void chip_spi_done(void *user_data, uint8_t *buffer, uint32_t count) {
    chip_state_t *chip = (chip_state_t*)user_data;
    if (!count) {
        // This means that we got here from spi_stop, and no data was received
        return;
    }

    // Read the analog input value, change to an 14-bit value, save in
    // the buffer.
    // The result will be read back during the next SPI transfer.
    float temp = ADC_full_scale*pin_adc_read(chip->a_in_pin);
    uint16_t itemp = (uint16_t) (temp);
    chip->spi_buffer[0] = itemp & 0x00FF;
    chip->spi_buffer[1] = (itemp >> 8) & 0x00FF;
}
```

main.c

- What is needed;
 - Change the task delay time to support the desired sample rate (10 Hz)
 - Add support for the ADC readings (temperature calculation)
 - Set up the SPI interface for the defaults of the SPI0 instance

main.c

- Add some include files to support the changes
- Change the fundamental delay time to 100 ms (1/10 Hz)
- Remove the producer-consumer tasks and make a task for getting the ADC data



28

```
/**
 * ECE 315 Computer Interfacing
 *
 * @file main.c
 * @details Connect a Pico to a custom SPI chip
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version 1.0
 * @licence MIT
 */
#include "main.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "pico/binary_info.h"
#include "hardware/spi.h"

#define BUF_LEN 0x02 // byte; a 16-bit ADC result is expected

// Set a delay time of exactly 100ms -- 10 Hz
const TickType_t ms_delay = 100 / portTICK_PERIOD_MS;

// References to the tasks
TaskHandle_t adc_task_handle = NULL;
```

main.c

- The ADC configuration struct contains only the sample period
- We will need to manually assert and de-assert the nCS signal
 - Make sure it's configured to be active low

```
/*
 * ADC task parameters struct
 */
typedef struct adc_config {
    TickType_t sample_period_ms;
} adc_config_t;

adc_config_t adc_config= {100 / portTICK_PERIOD_MS,};

#ifdef PICO_DEFAULT_SPI_CSN_PIN
static inline void cs_select() {
    asm volatile("nop \n nop \n nop");
    gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 0); // Active low
    asm volatile("nop \n nop \n nop");
}

static inline void cs_deselect() {
    asm volatile("nop \n nop \n nop");
    gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 1);
    asm volatile("nop \n nop \n nop");
}
#endif
```

main.c

- The #if check ensures we have included the SPI interface definitions
- 500 kHz SCK is more than fast enough to get the 16 bits
- Set the SPI default pin functions, but
- Set nCS pin to be plain old GPIO
- BUF_LEN is 2 bytes, 16 bits
- Set some conversion constants



30

```
/**
 * @brief The ADC task. Configure the SPI interface,
 * then read the ADC 16-bit value every
 * sample_period_ms milliseconds
 */
void adc_task(void* params) {

    adc_config_t cfg= *((adc_config_t*) params);

    #if !defined(spi_default) || !defined(PICO_DEFAULT_SPI_SCK_PIN) || !defined(PICO_DE
    #warning spi/spi_master example requires a board with SPI pins
    printf("Default SPI pins were not defined\n");
    #else
    spi_init(spi_default, 500 * 1000); // SPI clock is 500 kHz
    gpio_set_function(PICO_DEFAULT_SPI_RX_PIN, GPIO_FUNC_SPI);
    gpio_set_function(PICO_DEFAULT_SPI_SCK_PIN, GPIO_FUNC_SPI);
    gpio_set_function(PICO_DEFAULT_SPI_TX_PIN, GPIO_FUNC_SPI);
    // We need to manage the CSN pin ourselves, so make it a plain old GPIO
    gpio_init(PICO_DEFAULT_SPI_CSN_PIN);
    gpio_set_dir(PICO_DEFAULT_SPI_CSN_PIN, GPIO_OUT);
    gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 1);

    uint8_t in_buf[BUF_LEN];

    const float ADC_full_scale = ((float) 0x3FFF) / 5.0;
    const float scale = (953./4.65 + 115./0.563569)/2.0;
    const float BETA = 3950; // should match the Beta Coefficient of the thermistor
```

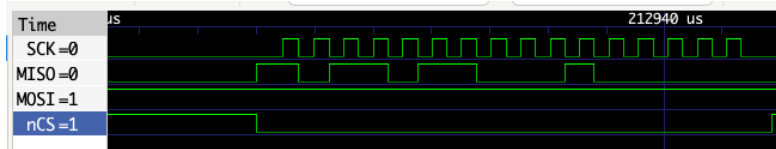
main.c

- Toggle nCS, read the SPI data
- Cast the two bytes in the buffer to an unsigned int
- If it's not zero, convert it to temperature
- Normally something else would be done with it, ...

```
while (true) {
    // Write the output buffer to MOSI, and at the same time read from MISO.
    cs_select();
    spi_read_blocking(spi_default, 0xff, in_buf, BUF_LEN);
    cs_deselect();

    // turn the received bits into the 14-bit ADC value
    uint16_t adc_val = ((in_buf[1]<<8) & 0x3F00) | in_buf[0];
    // check to be sure we don't process a startup or spurious value
    if (adc_val > 0) {
        printf("adc_val 0x%04x\n", adc_val);
        printf("adc_val %ld\n", adc_val);
        float adc_f = (double) (adc_val / ADC_full_scale * scale);
        // Convert the scaled adc value to degrees Celsius
        float celsius = (1.0 / (log(1.0 / (1023. / adc_f - 1.0)) / BETA + 1.0 / 298.15) - 273.15);
        printf("celsius %ld\n", (long) celsius);
    }
    vTaskDelay(cfg.sample_period_ms);
}
```

#endif



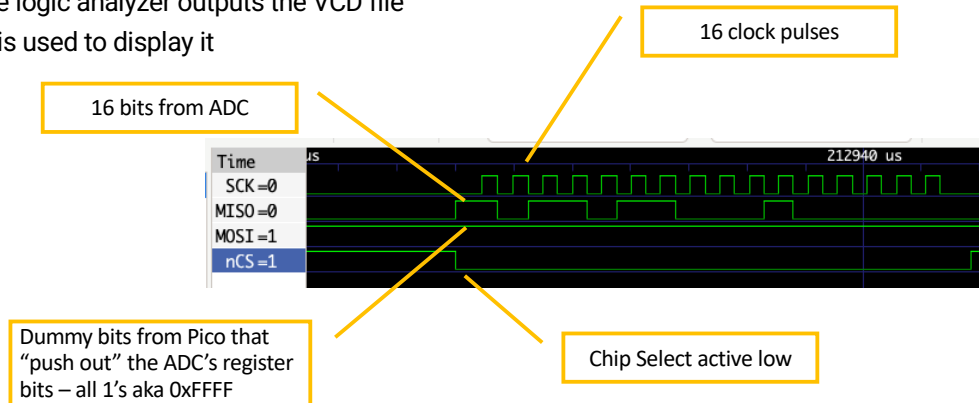
main.c

- The main() function is simple enough
- Make the task
- Start the task

```
/*  
 * RUNTIME START  
 */  
int main() {  
  
    // Enable STDIO  
    stdio_init_all();  
  
    // Set up two tasks  
    BaseType_t adc_status =  
        xTaskCreate(adc_task, "ADC_TASK", 128, &adc_config, 1, &adc_task_handle);  
  
    // Start the FreeRTOS scheduler  
    // Only proceed with valid tasks  
    if (adc_status == pdPASS) {  
        vTaskStartScheduler();  
    }  
  
    // We should never get here, but just in case...  
    while(true) {  
        // NOP  
    }  
}
```


main.c

- After running, the logic analyzer outputs the VCD file
- Here, GTKWave is used to display it





Next Lecture

- Inter-Integrated Circuit (I²C)

Questions

